

IT213 Lab 5: Transport Layer & Packet Inspection

Curl, Tcpdump, and the Art of Grep

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

Today's Mission

In previous labs, we looked at the **Phonebook** (DNS) and the **Roads** (Ping).

Today, we look at the **Trucks** (TCP) that carry the cargo.

- 1 **Bridge:** Verify our target (Gaia) is alive.
- 2 **Transfer:** Download a file using the command line ('curl').
- 3 **Inspect:** Capture the traffic and filter it using **Regex**.

The Target:

`http://gaia.cs.umass.edu/wireshark-labs/alice.txt`

Part 1: Manual File Transfer

We will use `curl` to act like a web browser without the GUI.

The Command:

```
curl http://gaia.cs.umass.edu/.../alice.txt -o alice.txt
```

Why use `-o`?

The `-o` flag saves the file to your disk. If we don't use it, `curl` dumps the text to the screen, which is messy!

Expected Output: Curl Statistics

When you run the command, you will see a **Progress Table**.

```
% Total % Received % Xferd Average Speed Time Time Time Current
      Dload Upload Total Spent Left Speed
100 152k 100 152k 0 0 356k 0 --:--:-- --:--:-- --:--:-- 360k
```

What to analyze:

- 1 **Total Size (152k):** This confirms we got the whole book (152 Kilobytes).
- 2 **Average Speed (356k):** This tells you the health of the connection.

New Tool: What is grep?

The Problem: Network traffic is massive. If you capture for 10 seconds, you might get 5,000 lines of text. How do you find the *one* packet you care about?

The Solution: `grep`

- Stands for: **G**lobal **R**egular **E**xpression **P**rint.
- It is a standard POSIX tool (Linux/Mac) that filters text.
- *Analogy:* It's like "Ctrl+F" but for millions of lines at once.

The Pipeline Concept: Huge Data → `grep "Pattern"` → Tiny Useful Data

What is Regex?

Regular Expressions (Regex) are a language for defining "Search Patterns."

Why is it hard? Computers use special characters. In Regex, brackets [] mean "Match ANY of these characters."

The Challenge for Today: We want to find the literal text: `Flags [S]`

The Escape Character (\)

Since [is special, we must "escape" it with a backslash.

Wrong: `grep "Flags [S]"` (Matches 'S' inside any flag)

Right: `grep "Flags \[S\]"` (Matches the literal bracket)

Part 2: The Command

We will capture traffic to a file, then filter it.

Step 1: Capture

```
sudo tcpdump -nv host gaia.cs.umass.edu > tcpdump.txt
```

(Note: The > symbol redirects the output to a file)

Step 2: Generate Traffic Run the curl command in a second window.

Step 3: Filter

```
grep "Flags \[\S\]" tcpdump.txt
```

Decoding the Output (Anatomy of a Packet)

Let's look at the EXACT line you will see:

```
10.0.0.29.51256 > 128.119.245.12.80: Flags [S], cksum 0x6653 (correct), seq  
3456998159, win 64240, length 0
```

Let's break it down:

- ❶ **10.0.0.29.51256**: Source IP + Source Port (Random Client Port).
- ❷ **> 128...80**: Destination IP + Port 80 (HTTP Server).
- ❸ **Flags [S]: SYN Flag**. This is the connection request!
- ❹ **seq 345...**: Random starting Sequence Number.
- ❺ **win 64240**: "I have room for 64,240 bytes of data."

Deep Dive: The Checksum (cksum)

Let's look at this specific piece: `cksum 0x6653` (correct)

1. What is 0x6653?

- The prefix `0x` tells us this is **Hexadecimal** (Base 16).
- It represents a 16-bit number calculated from the header data.

2. What is a Checksum?

- It is a "Digital Verification Seal."
- The Sender performs a math calculation on all the bits ($A + B + C = Sum$) and writes the **Sum** here.

3. Why does it say "(correct)"?

- `tcpdump` performed the same math on the packet it just captured.
- It compared its result with the number inside the packet.
- **They Matched!** This proves the data was NOT corrupted by a bad cable or electrical noise during travel.

Part 3: The 3-Way Handshake (Visual)

Finally, we will verify this in Wireshark. **The Goal:** Find the first 3 packets (Green/Grey).

No.	Time	Source	Destination	Info
1	0.00	192.168...	128.119...	54321 → 80 [SYN] Seq=0 ...
2	0.04	128.119...	192.168...	80 → 54321 [SYN, ACK] Seq=0 ...
3	0.04	192.168...	128.119...	54321 → 80 [ACK] Seq=1 ...

The Checklist:

- 1 **Packet 1:** [SYN] → "Can we talk?"
- 2 **Packet 2:** [SYN, ACK] → "Yes! I acknowledge you."
- 3 **Packet 3:** [ACK] → "Great, let's go."

Once you find these 3 packets, you are done!

The Mystery Flag: [P] (Push)

You might see a flag like this in your output:

```
Flags [P.], seq 1:50, ack 100, win 500, length 49
```

What is [P]?

- **Name: PSH** (Push).
- **Meaning:** "Don't wait! Send this data to the application immediately."

The Waiter Analogy

Normal TCP: The waiter (TCP) waits until his tray is full of dishes (Data) before walking to the kitchen. This is efficient.

TCP with [P]: A customer puts one dish on the tray and yells "GO!" The waiter runs to the kitchen immediately without waiting for more.

Why do you see it?

- **Chat Apps / SSH:** When you type 'A', you want it sent *now*, not